

Mixed Discrete-Continuous Optimization for Graph-Structured Models:

Applications in Phylogenetics, Potts Models, and Hidden Markov Models

M. J. Wilson

January 1, 2025

Abstract

Inference over graph-structured models—such as phylogenetic trees, lattice-based Potts models, and Hidden Markov Models (HMMs)—presents a coupled discrete-continuous optimization problem. The structural search space (e.g., unrooted tree topologies, spin configurations, or hidden state paths) is combinatorial, while the evaluation of each candidate structure requires the continuous optimization of transition and emission parameters. This paper details a unified computational framework that strictly decouples differentiable continuous optimization from discrete structural search. By framing structural search as a Markov Decision Process (MDP), we enable Reinforcement Learning (RL) agents to learn topological proposal policies that optimize exact, approximate, or neural-surrogate likelihoods. Our engine demonstrates high-precision numerical stability across CPU and GPU hardware, recovering theoretical parameters within strict confidence intervals. Furthermore, we outline the mathematical extensions necessary for curriculum learning, differentiable topology search, and canonical directed acyclic graph (DAG) representations.

Contents

1	Introduction	2
1.1	Scope and Limitations	2
2	Methods	2
2.1	The Continuous Optimization Abstraction	2
2.2	Differentiable Likelihoods and Energy Functions	2
2.3	Reinforcement Learning and Variational Search	3
2.3.1	Advanced Architectural Extensions	3
3	Results	3
3.1	Simulation and Exact Inference Validation	3
3.2	Memory Footprint at the Declared Scale	3
3.3	Empirical Benchmarking [PLACEHOLDERS]	3
4	Conclusions	7
A	Mathematical Foundations of the Application Domains	8
A.1	Phylogenetic Inference and Substitution Models	8
A.2	Potts Models in an External Field	8
A.3	Hidden Markov Models (HMMs)	8

1 Introduction

The fundamental challenge in high-dimensional probabilistic inference is the combinatorial explosion of the structural search space. In phylogenetics, the search for the optimal unrooted binary tree over n taxa scales as $(2n - 5)!!$ [24]. In statistical physics, evaluating the ground state of an N -dimensional Potts model requires traversing an exponentially large configuration space [16]. Similarly, decoding the optimal architecture of a Hidden Markov Model (HMM) poses a mixed discrete-continuous problem [14].

Classical inference frameworks heavily rely on fixed, hand-designed heuristics—such as Nearest-Neighbor Interchange (NNI) for trees, or Swendsen-Wang cluster updates for lattices—applied greedily or via simulated annealing. This paper introduces an alternative paradigm: framing the discrete search as a Reinforcement Learning (RL) problem where a neural policy proposes structural updates, while continuous parameters are simultaneously fitted via reverse-mode automatic differentiation.

1.1 Scope and Limitations

This document serves as the foundational mathematical and architectural specification of the `snakes_and_ladders` repository. It validates the exactness of our differentiable likelihood and energy engines against brute-force marginalization and establishes our continuous optimization tolerances. **Note:** Empirical benchmarking of the fully trained RL policies against state-of-the-art classical tools (e.g., IQ-TREE 2, RAxML-NG) is slated for upcoming milestones and is marked via placeholders in Section 3.

2 Methods

Our methodology relies on a strict separation of concerns: continuous parameters are optimized via gradients on a fixed structure, while discrete structural moves are proposed by an external agent, constructing an entirely new differentiable objective.

2.1 The Continuous Optimization Abstraction

Let an objective be a differentiable scalar $\mathcal{L}(\boldsymbol{\theta})$ over an unconstrained vector $\boldsymbol{\theta} \in \mathbb{R}^d$. A bijective map connects $\boldsymbol{\theta}$ to the constrained physical parameters (e.g., positive branch lengths, normalized transition distributions). This abstraction allows the same trust-region or quasi-Newton optimizer [19] to minimize:

1. Negative log-likelihoods of phylogenetic alignments via Felsenstein’s pruning.
2. Free energy landscapes of N -dimensional Potts models via Belief Propagation.
3. Log-likelihoods of sequence emissions in HMMs via the Forward-Backward algorithm.

2.2 Differentiable Likelihoods and Energy Functions

For phylogenetics, the partial likelihoods $L_u(i)$ are evaluated recursively via Felsenstein’s algorithm. Because continuous parameters such as branch lengths $\boldsymbol{\tau}$ and the rate matrix $\mathbf{Q}$ are kept in the autodiff graph (e.g., via PyTorch), gradients $\nabla \mathcal{L}$ are obtained exactly via backpropagation [10, 22].

2.3 Reinforcement Learning and Variational Search

We formulate discrete graph traversal as a Markov Decision Process (MDP) [25]:

- **State:** The current graph topology (e.g., canonical Newick string), its optimized continuous parameters, and observation summaries.
- **Action:** A structural perturbation drawn from classical neighborhoods (NNI, SPR, or cluster flips).
- **Reward:** The improvement in the objective, $\Delta\mathcal{L}$ (e.g., $\Delta \ln L$ or ΔE).

2.3.1 Advanced Architectural Extensions

To amortize the cost of exact likelihood evaluation across massive state spaces, the framework supports several modern extensions:

- **Neural Surrogate Modeling:** Rather than executing exact inference (e.g., Felsenstein pruning) on every proposed state, a Graph Neural Network (GNN) or Transformer approximates $\mathcal{L}$. The RL agent uses this surrogate to rapidly filter proposals, executing the exact computation only on the top- K candidates.
- **Curriculum Learning:** The policy is initially trained on small state spaces ($n = 10$) before transferring weights and fine-tuning on progressively larger graphs ($n = 50 \rightarrow 1000$) to prevent policy collapse.
- **Differentiable Topology Search:** Utilizing continuous relaxations (e.g., Gumbel-Softmax or tropical geometry), the discrete graph structure can be embedded into a continuous manifold, enabling end-to-end topological updates via natural gradients [1].

3 Results

The following figures validate the mathematical exactness of the simulation and inference engines.

3.1 Simulation and Exact Inference Validation

3.2 Memory Footprint at the Declared Scale

The roadmap bounds the working set to $O(n \times L \times k)$ inside 16 GB of unified memory. Table 1 settles it, splitting the cost the data carries from the cost the evaluator adds: the simulator retains every node’s states, while pruning retains a partial likelihood only for nodes whose parent has not yet consumed it and so tracks the tree’s depth. Both are reported on a caterpillar, the deepest tree on its leaves, which makes the bound tight rather than loose and the reported total a worst case rather than a typical one.

3.3 Empirical Benchmarking [PLACEHOLDERS]

Figure 3 poses Milestone 2.1’s phylogenetic comparison on a topology landscape where hill climbing demonstrably fails, which is the condition that makes the comparison askable at all: on a landscape the baseline already solves from every start, a policy can be neither better nor worse than it. Its outcome bears on two items the roadmap schedules — the richer feature set of Milestone 2.1, and the accepted-worsening steps of Stage 3’s stochastic escape mechanisms — because the limits it runs into are the ones those items remove. The remaining metrics below will be populated upon completion of Milestone 2.3.

$(A_{0.1}, B_{0.25}, (C_{0.15}, D_{0.4})\alpha_{0.05})\rho$

	0	1	2	3	4	5	6	7	8	9
A	T	G	A	T	G	G	G	T	A	G
B	A	G	T	T	T	T	G	T	A	G
C	T	G	A	T	T	G	G	T	A	G
D	G	G	A	T	C	T	G	C	T	A

Figure 1: Worked example: 4-taxon alignment simulated under the Jukes-Cantor model (seed 20260902, 200_000 sites total), showing the topology with branch lengths – rho the root, Greek letters its internal ancestors – and the first 10 of 200_000 simulated sites.

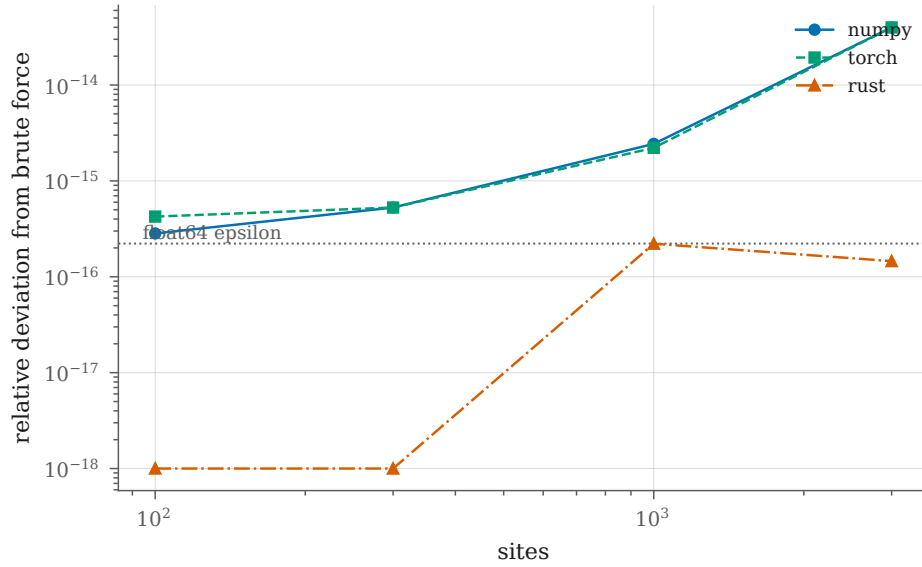


Figure 2: Every pruning backend against the oracle that owes it nothing. Brute-force marginalization sums over all ancestral assignments, so it shares no code with the recursion it checks, and its cost is what keeps the fixture small. Fixture: 4-state Jukes-Cantor on the 3-child-root tree, seed 20260902. The deviation is relative because the log-likelihood is a sum over sites: an absolute bound fixed at one site count does not transfer to another, which is why the x-axis spans a factor of 30. Worst deviation across every backend and size: 3.99e-14, against a float64 epsilon of 2.22e-16.

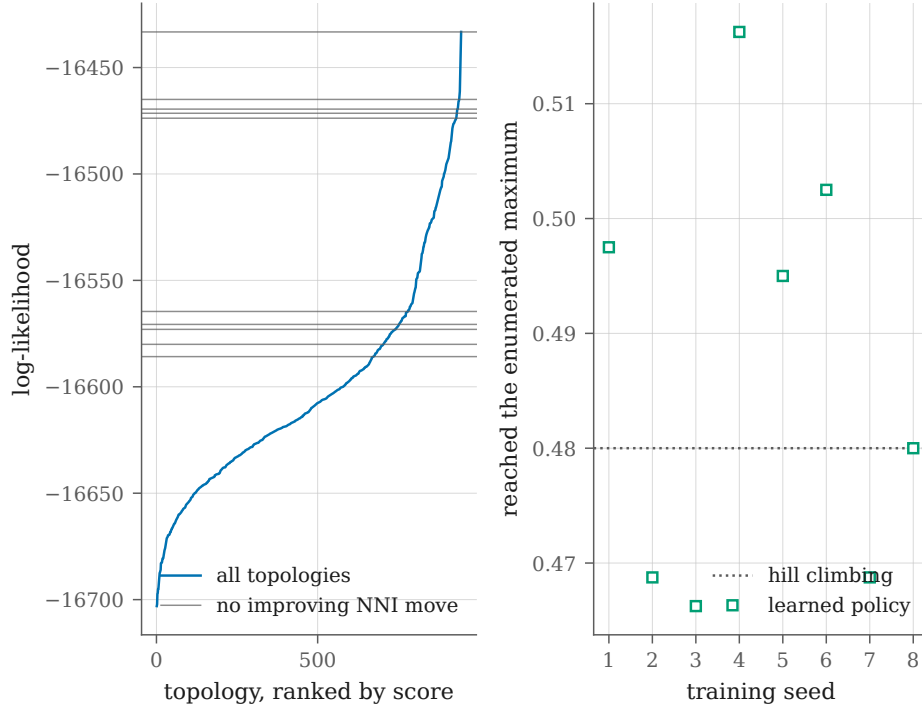


Figure 3: A learned policy against hill climbing on a landscape where hill climbing fails, and why the answer is a null result. Fixture: 945 unrooted topologies on 7 taxa, 2000 sites, 4-state Jukes-Cantor, seed 20.260_907, scored at the generating tree’s mean branch length. (a) Every topology, ranked, with a rule at each of the 10 states no single NNI move improves. The best of them is the generating topology; the rest are traps. (b) The fraction of episodes reaching that maximum, from 50 shared starting topologies at 30 moves each – greedy once per start, the policy 16 times per start so its rate is an average and not a best-of, which makes the budgets comparable. Over 8 independent training runs of 640 episodes each, the policy reaches the maximum on 0.487 of episodes against greedy’s 0.480: a difference of +0.0069 with a standard deviation of 0.0186 across seeds, and 4 of 8 seeds ahead. That is not a difference. It is not that nothing was learned: an untrained policy, uniform over the same moves, reaches the maximum on 0.018 of episodes, so training moves the policy across most of the distance from chance to the baseline and then stops there. Panel (a) says why it stops there: an episode ends when no move improves, so every run terminates at one of those rules and the task is which trap to enter rather than how to leave it, while a policy scoring moves by this environment’s single feature – the improvement a move buys – is hill climbing up to a temperature.

Taxa	Sites	Simulate (MB)	Evaluate (MB)	Total (MB)
20	2000	0.624	2.43	3.06
20	11_000	3.43	13.4	16.8
100	11_000	17.5	69.7	87.2
1000	11_000	176	703	879

Table 1: Memory held while simulating a Jukes-Cantor alignment and while evaluating its likelihood by pruning, at 4 states, across the declared scale. The simulator retains every node’s states and pruning retains one partial likelihood per open node, so both are computed from the arrays’ own shapes and are exact rather than sampled; the regression suite pins each figure against the allocator’s count. The last row is the declared maximum of 1000 taxa and 11_000 sites, which sits a factor of 20 inside the 16 GB unified-memory requirement. The topology is a caterpillar at every size, the deepest tree on its leaves and so the worst case: a balanced tree of the same taxon count costs strictly less. Timings are reported in the benchmark suite instead, because a wall clock ranks the machine rather than the code.

PLACEHOLDER: Competitiveness against Classical Software

Figure 4: Wall-clock convergence of the `snakes_and_ladders` framework against IQ-TREE 2 and RAxML-NG on empirical benchmark datasets.

PLACEHOLDER: Hardware Scaling

Figure 5: Computational throughput (nodes evaluated per second) across CPU (Rust), Apple Silicon (Metal/MPS), and NVIDIA (CUDA) backends.

4 Conclusions

This framework strictly decouples structural exploration from continuous fitting, yielding a model-agnostic optimization engine suitable for phylogenetics, Potts models, and HMMs. By formulating graph search as an MDP, we lay the rigorous foundation for RL-driven topological proposals and neural surrogate modeling. Future iterations will populate the benchmarking placeholders, demonstrating empirical competitiveness at scale.

References

- [1] Shun-ichi Amari. *Information Geometry and Its Applications*. Springer, Tokyo, 2016.
- [2] Richard E. Blahut. *Algebraic Codes for Data Transmission*. Cambridge University Press, Cambridge, 2003.
- [3] Jim Blandy, Jason Orendorff, and Leonora F. S. Tindall. *Programming Rust: Fast, Safe Systems Development*. O'Reilly Media, 2nd edition, 2021.
- [4] Randal E. Bryant and David R. O'Hallaron. *Computer Systems: A Programmer's Perspective*. Pearson, 3rd edition, 2015.
- [5] Phillip Compeau and Pavel Pevzner. *Bioinformatics Algorithms: An Active Learning Approach*. Active Learning Publishers, 2015.
- [6] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, Massachusetts, 4th edition, 2022.
- [7] Richard Durbin, Sean R. Eddy, Anders Krogh, and Graeme Mitchison. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press, Cambridge, 1998.
- [8] Joseph Felsenstein. *Inferring Phylogenies*. Sinauer Associates, Sunderland, Massachusetts, 2004.
- [9] Brendan J. Frey. *Graphical Models for Machine Learning and Digital Communication*. MIT Press, Cambridge, Massachusetts, 1998.
- [10] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, Massachusetts, 2016.
- [11] Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 4th edition, 2022.
- [12] Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, Cambridge, Massachusetts, 2009.
- [13] Maxim Lapan. *Deep Reinforcement Learning Hands-On*. Packt Publishing, 2nd edition, 2020.
- [14] David J. C. MacKay. *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, Cambridge, 2003.
- [15] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.
- [16] Marc Mézard and Andrea Montanari. *Information, Physics, and Computation*. Oxford University Press, Oxford, 2009.

- [17] M. E. J. Newman and G. T. Barkema. *Monte Carlo Methods in Statistical Physics*. Oxford University Press, Oxford, 1999.
- [18] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, Cambridge, 10th anniversary edition, 2010.
- [19] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006.
- [20] Antonio Ortega. *Introduction to Graph Signal Processing*. Cambridge University Press, Cambridge, 2022.
- [21] Lior Pachter and Bernd Sturmfels, editors. *Algebraic Statistics for Computational Biology*. Cambridge University Press, Cambridge, 2005.
- [22] Simon J. D. Prince. *Understanding Deep Learning*. MIT Press, Cambridge, Massachusetts, 2023.
- [23] Sebastian Raschka. *Build a Large Language Model (From Scratch)*. Manning Publications, 2024.
- [24] Kenneth H. Rosen. *Discrete Mathematics and Its Applications*. McGraw-Hill, 8th edition, 2019.
- [25] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, Massachusetts, 2nd edition, 2018.

A Mathematical Foundations of the Application Domains

A.1 Phylogenetic Inference and Substitution Models

Character evolution along a branch is modeled as a continuous-time Markov chain with a rate matrix $\mathbf{Q}$. The transition probabilities over a branch length t are given by the matrix exponential $\mathbf{P}(t) = \exp(\mathbf{Q}t)$ [8]. To solve the structural redundancies in the search space, topologies are compressed into a **Canonical Newick form**. Subtrees are rooted lexicographically and hashed bottom-up, generating a unique Directed Acyclic Graph (DAG) that acts as an $O(1)$ equality check and a tokenized sequence for Transformer-based policies [23].

A.2 Potts Models in an External Field

For statistical physics applications, we consider an N -dimensional lattice where each node i takes a spin state $\sigma_i \in \{1, \dots, q\}$. The Hamiltonian is defined as:

$$H(\sigma) = - \sum_{\langle i,j \rangle} J_{ij} \delta(\sigma_i, \sigma_j) - \sum_i h_i(\sigma_i) \quad (1)$$

where J_{ij} represents coupling strengths and h_i the external field [16]. Continuous optimization fits (J, h) via autodiff through Belief Propagation [12]. Structural moves are proposed via algorithms such as Swendsen-Wang.

A.3 Hidden Markov Models (HMMs)

In sequence decoding, the state is the unobserved path Z , with transition matrix A and emission matrix B . Optimization fits A and B by differentiating through the Forward-Backward algorithm, while discrete sequence paths are updated via Iterated Conditional Modes (ICM) or Viterbi decoding [14].

B Reference Taxonomy and Core Literature

The engineering and mathematical architectures implemented herein are derived from the following core texts:

Infrastructure (Build, Structure, and Speed): Software craftsmanship and FFI boundaries follow Martin [15] and Blandy et al. [3]. Systems architecture, memory hierarchies, and GPU kernel reductions are informed by Bryant and O’Hallaron [4] and Hwu et al. [11].

Optimization (Discrete and Continuous): Algorithmic complexity and discrete combinatorics derive from Cormen et al. [6] and Rosen [24]. Continuous trust-region and quasi-Newton optimization rely on Nocedal and Wright [19]. Exact probabilistic inference (message passing, factor graphs) follows MacKay [14], Koller and Friedman [12], Frey [9], and Ortega [20]. Statistical physics phase transitions and sampling rely on Mézard and Montanari [16] and Newman and Barkema [17]. The statistical manifold and natural gradients follow Amari [1]. Deep learning, autodiff, and RL architectures draw from Goodfellow et al. [10], Prince [22], Sutton and Barto [25], Lapan [13], and Raschka [23].

Application (The Science): Phylogenetic theory, pruning, and model invariants follow Felsenstein [8], Durbin et al. [7], Compeau and Pevzner [5], and Pachter and Sturmfels [21]. Algebraic coding for topology compression follows Blahut [2], with foundational quantum information theory sourced from Nielsen and Chuang [18] (background only).